

MAULANA AZAD NATIONAL INSTITUTE OF TECHNOLOGY - BHOPAL



DEPARTMENT OF MATHEMATICS BIOINFORMATICS AND COMPUTER APPLICATION

PROJECT GUIDES:

Dr. SANJAY SHARMA

Dr. SHUBHANKAR DUTTA

RASHI GUPTA 25204031105

VISHALI 25204031113

ABHIJEET SAIN 25204031121

ANCHAL JAISWAL 25204031130

PUSHPENDRA 25204031138

RAVI KUMAR MAHTO 25204031146

Table of Contents -

1 .	Abstract
2.	Scope
3.	Objective
4.	Tools and technologies used
5.	Data flow diagram
6.	Entity relationship diagram
7.	Normalization
8.	Data dictionary
9.	Views
10.	Triggers
11.	Procedures and functions
12.	Relationship, cascading, and cardinality
9.	Queries
10.	Conclusion

ABSTRACT

WHY THIS PROJECT?

The major problem in traditional supermarket management is the lack of an efficient system to track inventory, sales, and customer data. Manual systems often lead to errors, data inconsistency, and difficulty in retrieving information quickly.

This project aims to solve these issues using a structured database system. In modern retail environments, managing inventory manually can lead to errors, stock mismanagement, and inefficiency.

A supermarket requires a reliable system to track product availability, sales transactions, and supplier details. The Supermarket Inventory System is developed to address these challenges by providing an organized and automated way of managing inventory and sales data using a database management system.

The project demonstrates the use of database concepts such as table creation, relationships, constraints, and SQL queries for data retrieval and management.

SCOPE OF THE PROJECT

Analytical Scope (The "Insights")

Predictive Logistics: Using the "Festival Demand" logic to identify seasonal trends and optimize stock levels before peak periods.

Technical / Architecture Scope (The "How")

- **Schema Normalization:** Designing a 3rd Normal Form (3NF) architecture to ensure data integrity and eliminate redundancy across 31 tables.
- **Automated Data Integrity:** Utilizing Triggers to enforce complex business rules (like preventing sales of expired items) that simple constraints cannot handle.

Security & Data Abstraction: Using Views to provide a "Need-to-Know" access model, ensuring sensitive data (like profit margins) is hidden from standard users.

Functional Scope

(The "What")

- **Inventory & Stock Management:** Comprehensive tracking of products across 31 tables, including batch-wise management and automated stock reconciliation.
- **Customer Relationship Management (CRM):** A tiered loyalty program that tracks purchase history and manages reward points automatically.

Operational Scope (The "Constraints")

- **Risk Mitigation:** Automated Low-Stock Alerts and system-level blocks on Expired Inventory via the trigger layer.

Logistics Strategy: Implementation of the FEFO (First-Expired, First-Out) methodology to minimize inventory wastage

OBJECTIVES OF THE PROJECT

RELIABILITY & INTEGRITY

- **Enforce Data Standards:** Keep the database in 3NF form to avoid Redundant data and maintain proper structure.
- **System-Level Validation:** Use strict triggers to stop errors like selling expired products.

AUTOMATION & EFFICIENCY

- **Make operations easier:** Automate billing and restocking processes to save time and reduce manual work.
- **Synchronize Data:** Keep all data updated instantly across all 31 tables using triggers, so that stock information is always correct, and there are no mismatches.

STRATEGIC INTELLIGENCE

- **Provide Actionable Insights:** To generate specialized management reports via Views, focusing on profitability, staff performance, and inventory health.
- **Forecasting & Planning:** To analyze historical sales data (Festival Demand) to help management prepare for peak shopping seasons.

CUSTOMER & RESOURCE MANAGEMENT

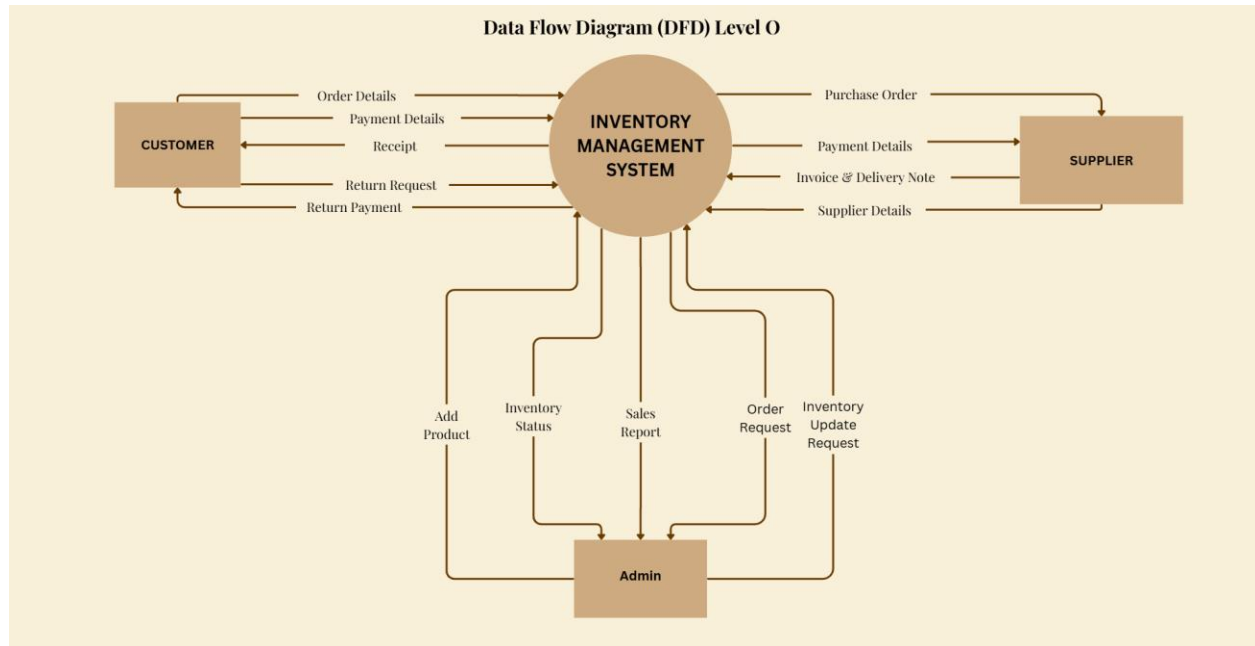
- **Personalize Experience:** Create a smart loyalty system that automatically tracks customer activity and rewards without manual work.
- **Optimize Logistics:** Reduce waste and old stock by using the FEFO (First-Expired, First-Out) method at the database level.

TOOLS & TECHNOLOGIES USED

- Database Platform: Supabase
- Database Engine: PostgreSQL
- Query Language: SQL (DDL, DML, PL/pgSQL)
- Interface: Supabase SQL Editor and Table Editor

Data flow diagram

DFD LEVEL – 0 FLOW EXPLANATION

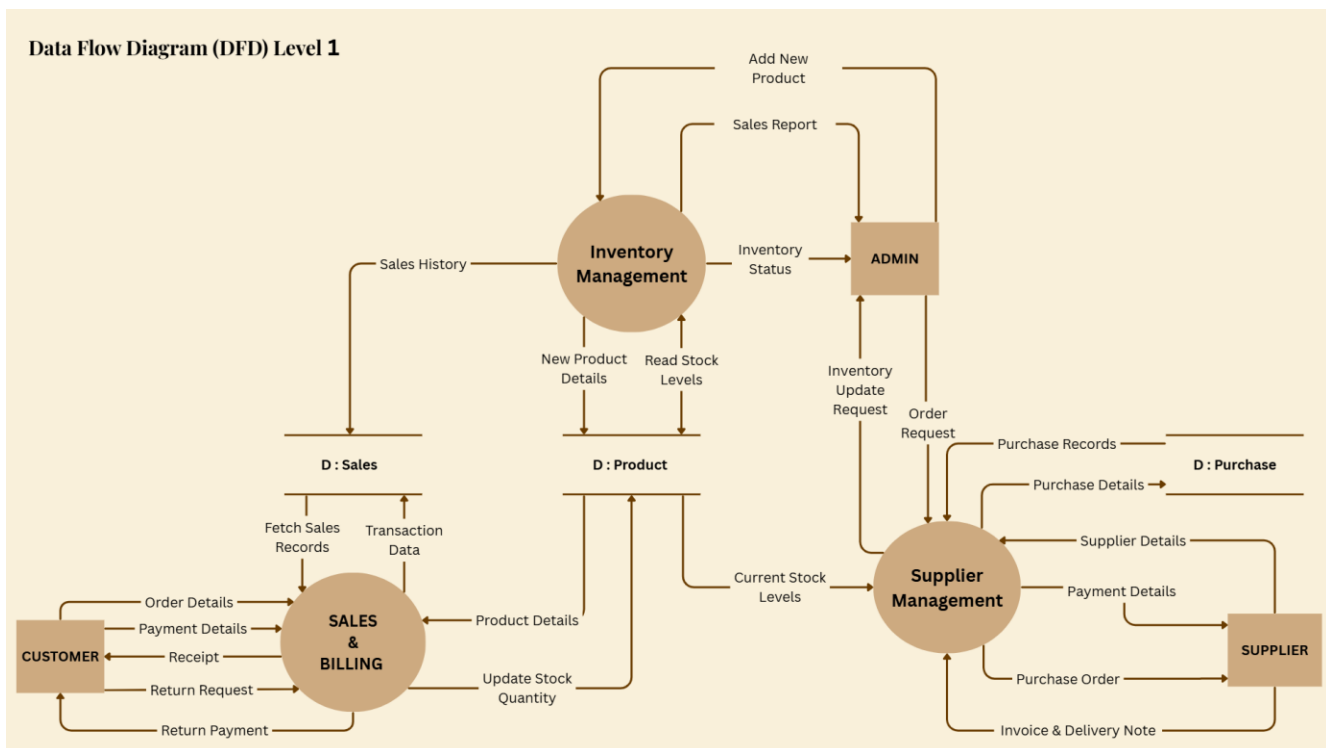


- The Inventory Management System acts as the central process connecting all entities.
- The **Customer** sends order details and payment information to the system.
- The system processes the request and provides receipts and handles return payments.
- Customers can also initiate return requests, which are processed by the system.
- The system sends purchase orders and payment details to the **Supplier**.
- The Supplier responds with supplier details, invoices, and delivery notes.
- The **Admin** interacts with the system to manage operations such as adding products and updating inventory.
- The Admin can also view inventory status and generate sales reports.

DFD Level 1 – Flow Explanation

- The system is divided into three main processes: **Sales & Billing**, **Inventory Management**, and **Supplier Management**.
- The **Customer** interacts with the Sales & Billing module by providing order and payment details and receiving receipts and return payments.
- Sales & Billing stores and retrieves transaction data from the **Sales data store**.
- Product details are fetched from the **Product data store**, and stock quantities are updated after each sale.

- The **Inventory Management** module maintains product information and stock levels.
- It reads and updates data in the Product data store and provides inventory status and sales reports to the Admin.
- The **Admin** monitors inventory, adds new products, and sends inventory update and order requests.
- The **Supplier Management** module handles purchase operations and interacts with the Supplier.
- It sends purchase orders and payment details to the Supplier and receives invoices, delivery notes, and supplier information.
- All purchase-related data is stored in the **Purchase data store**.
- Overall, the system ensures smooth coordination between sales, inventory, and supplier operations while maintaining proper data flow across all modules.

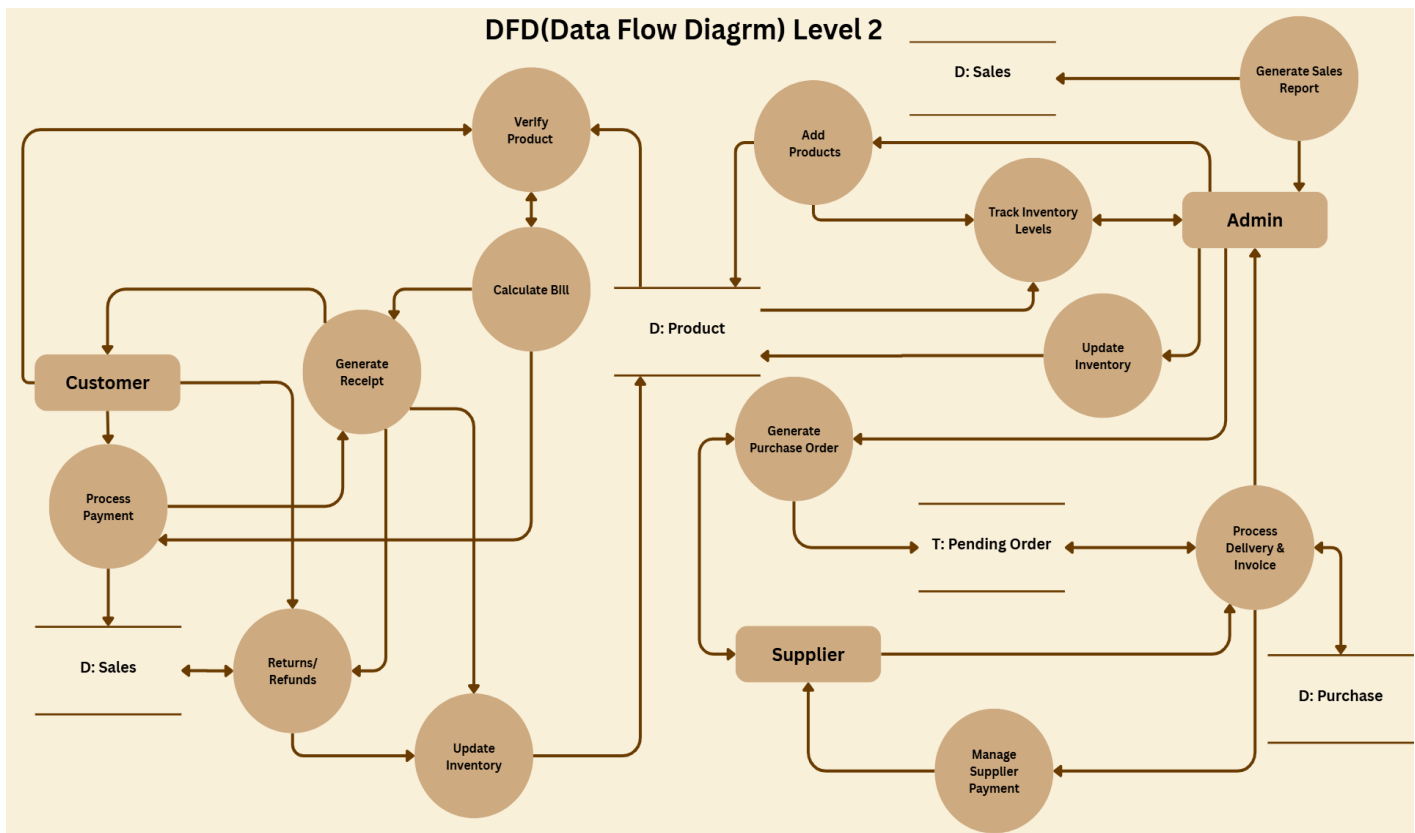


DFD Level 2 – Flow Explanation

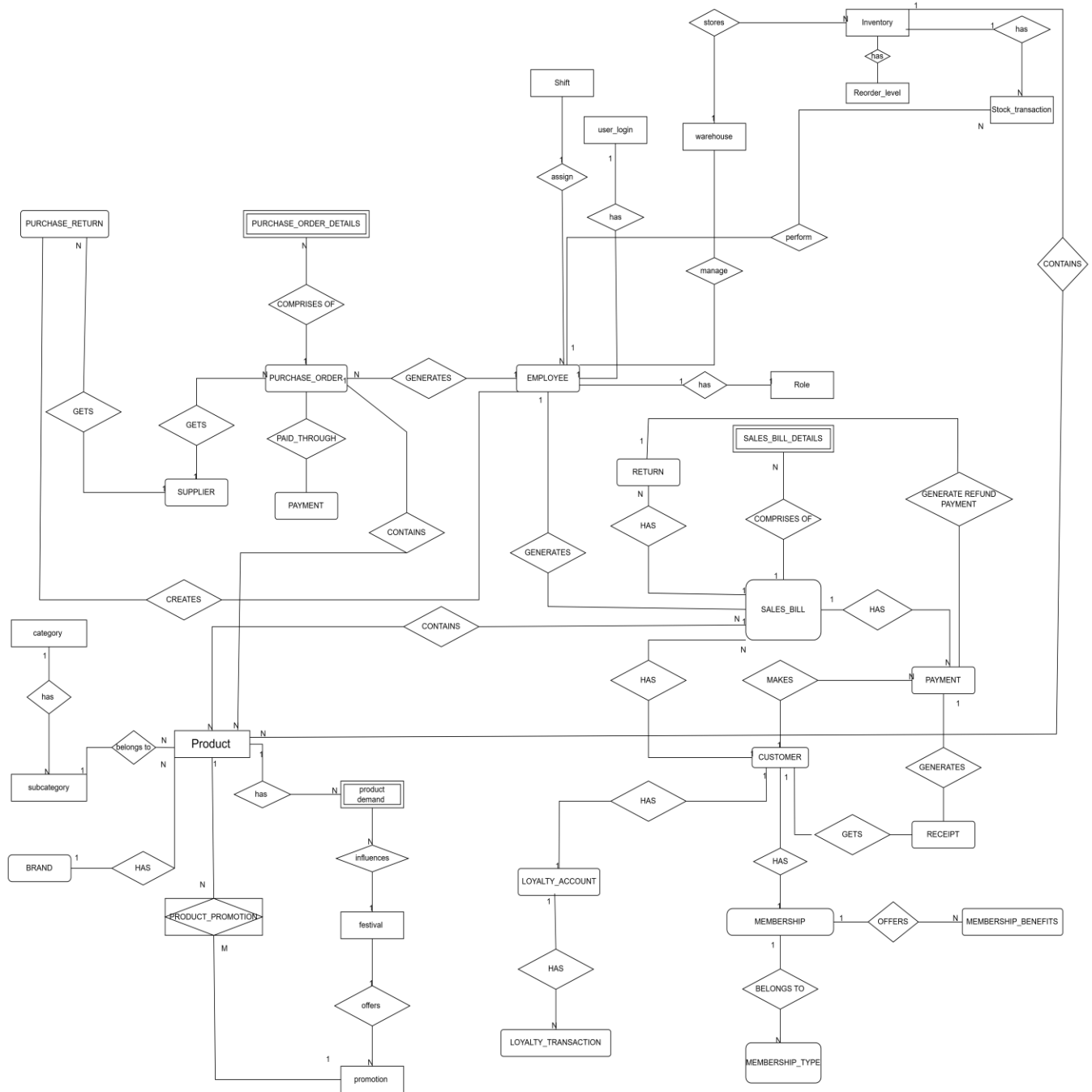
1. Sales & Billing Process
2. Inventory Management Process
3. Supplier Management Process

- The system is further broken down into detailed processes such as product verification, billing, payment processing, inventory updates, and supplier management.

- The **Customer** initiates the process by placing an order and making a payment, which is handled by the payment processing module.
- The system verifies product details and calculates the bill before generating a receipt for the customer.
- All sales transactions are stored in the **Sales data store** for record-keeping and reporting.
- In case of returns, the system processes refunds and updates inventory accordingly.
- The **Inventory module** continuously updates stock levels based on sales and returns.
- The **Admin** manages operations like adding products, tracking inventory, updating stock, and generating sales reports.
- When stock levels are low, the system generates purchase orders and stores them as pending orders.
- The **Supplier** receives purchase orders and provides delivery along with invoice details.
- The system processes delivery and invoices, updates the **Purchase data store**, and manages supplier payments.
- Overall, the system ensures detailed and automated handling of sales, inventory, and supplier processes with proper data storage and flow.



ER DIAGRAM (ONLY ENTITIES)



NORMALIZATION

The database has been successfully normalized up to **Third Normal Form (3NF)** to eliminate redundancy and ensure data integrity.

- **1NF:** Ensured atomic attributes and removed repeating groups by separating sales into **SALES_BILL** and **SALES_BILL_DETAILS**, with proper primary keys.
- **2NF:** Eliminated partial dependency by storing product details in the **PRODUCT** table and keeping only transaction-specific data in **SALES_BILL_DETAILS**.
- **3NF:** Removed transitive dependencies by separating related data into independent tables such as **MEMBERSHIP_TYPE**, **PAYMENT**, **SUPPLIER**, and **INVENTORY**, and linking them using foreign keys.

EXPLANATION

- Customer-related data is separated from membership details by creating a dedicated **MEMBERSHIP_TYPE** table, reducing repetition of membership information.
- Sales data is structured into two tables: **SALES_BILL** (main transaction) and **SALES_BILL_DETAILS** (product-wise details), avoiding duplication of product data in each bill.
- Product and inventory information are maintained in separate tables, allowing independent updates of stock without affecting product details.
- Supplier and purchase-related data are divided into **SUPPLIER**, **PURCHASE**, and related tables to ensure organized storage of procurement information.
- Payment information is stored separately from sales transactions, enabling flexible handling of multiple payment methods and records.
- All tables are connected using **primary keys and foreign keys**, ensuring proper relationships without redundant data storage.
- Redundant attributes are removed, and each table stores only relevant data, improving consistency and maintainability.

Outcome

- Eliminates data redundancy
- Improves data consistency and integrity
- Simplifies data maintenance and updates
- Enhances overall database performance

DATA DICTIONARY

Modules	Tables	Numbers of tables	Purpose
Product module	Product, brand, category, sub-category, inventory, warehouse	6	Manages product information, classification, and stock. organizes products into categories and tracks their availability in warehouses through inventory.
Supplier module	Supplier, purchase order,purchase_order _detail, purchase_return	4	Handles supplier details and purchasing process, including order placement, item tracking, and return of defective goods.
Sales and billing module	Sales_bill, sales_bill_detail, sales_return, receipt, payment, Customer, loyalty_account, loyalty_transaction	8	Manages sales transactions, billing, payments, and returns. Also maintains customer data and loyalty points system.
Employee module	Employee, role, shift, user-login, stock_transaction, reorder_level	6	Manages employee data, roles, shifts, and system access. Also tracks stock movement and maintains reorder levels.
Profit and marketing module	Product-demand, promotion, product-promotion, membership, membership benfits, membership type,festival	7	Supports marketing and analysis through promotions, demand tracking, and membership benefits to improve customer engagement.

1. PRODUCT

Column Name	Data Type	Key	Constraints	Description
product_id	INT	PK		Unique product
name	VARCHAR		NOT NULL	Product name
current_price	DECIMAL		CHECK > 0	Current selling price
sub_category_id	INT	FK	NOT NULL	References SUB_CATEGORY
brand_id	INT	FK		References BRAND

2. CATEGORY

Column Name	Data Type	Key	Constraints	Description
category_id	INT	PK		Unique category ID
name	VARCHAR	UK	NOT NULL	Category name

3. SUB_CATEGORY

Column Name	Data Type	Key	Constraints	Description
sub_category_id	INT	PK		Unique subcategory
name	VARCHAR		NOT NULL	Subcategory name
category_id	INT	FK	NOT NULL	References CATEGORY

4. BRAND

Column Name	Data Type	Key	Constraints	Description
brand_id	INT	PK		Brand ID
brand_name	VARCHAR	UK	UNIQUE, NOT NULL	Brand name

5. INVENTORY

Column Name	Data Type	Key	Constraints	Description
inventory_id	SERIAL	PK		Unique inventory ID
product_id	INT	FK	NOT NULL	Product reference
warehouse_id	INT	FK	NOT NULL	Warehouse reference
batch_number	VARCHAR		NOT NULL	Batch identifier
mfg_date	DATE			Manufacturing date

expiry_date	DATE		CHECK valid	Expiry date
quantity_available	INT		≥ 0	Stock quantity
last_updated	DATE		DEFAULT CURRENT_DATE	Last update

6. WAREHOUSE

Column Name	Data Type	Key	Constraints	Description
warehouse_id	SERIAL	PK		Warehouse ID
warehouse_name	VARCHAR	UK	UNIQUE, NOT NULL	Name
location	VARCHAR		NOT NULL	Location
capacity	INT		> 0	Capacity
employee_id	INT	FK		Manager

7. STOCK_TRANSACTION

Column Name	Data Type	Key	Constraints	Description
transaction_id	SERIAL	PK		Transaction ID
transaction_type	VARCHAR		IN/OUT/TRANSFER	Type of movement
quantity	INT		> 0	Quantity moved
transaction_date	TIMESTAMP		DEFAULT NOW	Date of transaction
inventory_id	INT	FK	NOT NULL	Inventory reference
employee_id	INT	FK	NOT NULL	Employee responsible
supplier_id	INT	FK		Supplier reference
remarks	TEXT			Additional notes

8. REORDER_LEVEL

Column Name	Data Type	Key	Constraints	Description
reorder_id	SERIAL	PK		ID
product_id	INT	FK	NOT NULL	Product
warehouse_id	INT	FK	NOT NULL	Warehouse
min_quantity	INT		> 0	Minimum
target_quantity	INT		> min	Target

9. SUPPLIER

Column Name	Data Type	Key	Constraints	Description
supplier_id	INT	PK		Supplier ID
supplier_name	VARCHAR		NOT NULL	Name
phone	VARCHAR	UK	UNIQUE, 10 digits	Contact
email	VARCHAR	UK	UNIQUE	Email
address	VARCHAR		NOT NULL	Address
city	VARCHAR		NOT NULL	City

10. PURCHASE_ORDER

Column Name	Data Type	Key	Constraints	Description
purchase_id	INT	PK		Order ID
order_date	DATE		NOT NULL	Order date
total_amount	DECIMAL		≥ 0	Total cost
expected_delivery_date	DATE		$\geq$ order_date	Expected delivery
status	VARCHAR		ENUM	Order status
supplier_id	INT	FK	NOT NULL	Supplier
employee_id	INT	FK	NOT NULL	Managed by

11. PURCHASE_ORDER_DETAILS

Column Name	Data Type	Key	Constraints	Description
purchase_id	INT	PK/FK		Order ID
product_id	INT	PK/FK		Product
quantity_ordered	INT		> 0	Ordered qty
received_quantity	INT		≥ 0	Received qty
unit_cost	DECIMAL		> 0	Cost per unit

12. PURCHASE_RETURN

Column Name	Data Type	Key	Constraints	Description
purchase_return_id	INT	PK		Return ID
return_date	DATE		DEFAULT CURRENT_DATE	Date
return_quantity	INT		> 0	Quantity
reason	TEXT			Reason
refund_amount	DECIMAL		≥ 0	Refund
product_id	INT	FK	NOT NULL	Product
purchase_id	INT	FK	NOT NULL	Order

employee_id	INT	FK	NOT NULL	Employee
-------------	-----	----	----------	----------

13. SALES_BILL

Column Name	Data Type	Key	Constraints	Description
bill_id	INT	PK		Unique bill
bill_date	TIMESTAMP		DEFAULT NOW	Bill timestamp
total_amount	DECIMAL			Total bill value
employee_id	INT	FK	NOT NULL	Salesperson
customer_id	INT	FK		Customer
payment_status	VARCHAR		ENUM values	Payment status

14. SALES_BILL_DETAILS

Column Name	Data Type	Key	Constraints	Description
bill_id	INT	PK/FK		Bill reference
product_id	INT	PK/FK		Product sold
quantity_sold	INT		> 0	Quantity sold
unit_price	DECIMAL		> 0	Price per unit
discount	DECIMAL		≥ 0	Discount applied

15. PAYMENT

Column Name	Data Type	Key	Constraints	Description
payment_id	INT	PK		Payment ID
payment_date	TIMESTAMP		DEFAULT NOW	Payment time
payment_method	VARCHAR		ENUM	Mode (UPI, Cash)
payment_status	VARCHAR		ENUM	Status
amount_paid	DECIMAL		> 0	Amount paid
bill_id	INT	FK		Linked bill

16. RECEIPT

Column Name	Data Type	Key	Constraints	Description
receipt_id	SERIAL	PK		Receipt ID
receipt_number	VARCHAR	UK	UNIQUE	Receipt No
receipt_date	DATE		DEFAULT CURRENT_DATE	Date

payment_id	INT	FK	UNIQUE	Payment
bill_id	INT	FK	UNIQUE	Bill

17. SALES_RETURN

Column Name	Data Type	Key	Constraints	Description
sales_return_id	INT	PK		Return ID
bill_id	INT	FK	NOT NULL	Bill
product_id	INT	FK	NOT NULL	Product
return_date	DATE		DEFAULT CURRENT_DATE	Date
return_quantity	INT		> 0	Quantity
refund_amount	DECIMAL		≥ 0	Refund
reason	TEXT			Reason
employee_id	INT	FK		Employee

18. CUSTOMER

Column Name	Data Type	Key	Constraints	Description
customer_id	INT	PK	NOT NULL	Unique ID for each customer
name	VARCHAR		NOT NULL	Customer's full name
email	VARCHAR		CHECK (valid format)	Customer email
phone	VARCHAR	UK	UNIQUE, CHECK (10 digits)	Customer contact number
registration_date	DATE		DEFAULT CURRENT_DATE	Date of registration

19. LOYALTY_ACCOUNT

Column Name	Data Type	Key	Constraints	Description
account_id	SERIAL	PK		Loyalty account
customer_id	INT	FK	UNIQUE	Linked customer
total_points	INT		≥ 0	Points balance
start_date	DATE		DEFAULT NOW	Start date
end_date	DATE			Expiry

20. LOYALTY_TRANSACTION

Column Name	Data Type	Key	Constraints	Description
loyalty_transaction_id	SERIAL	PK		Transaction ID
account_id	INT	FK	NOT NULL	Loyalty account
bill_id	INT	FK		Related bill
transaction_date	TIMESTAMP		DEFAULT NOW	Date
points_earned	INT		≥ 0	Points added
points_redeemed	INT		≥ 0	Points used
transaction_type	VARCHAR		ENUM	Earn/Redeem
remarks	TEXT			Notes

21. MEMBERSHIP_TYPE

Column Name	Data Type	Key	Constraints	Description
membership_type	VARCHAR	PK		Type name

22. MEMBERSHIP_BENEFITS

Column Name	Data Type	Key	Constraints	Description
benefit_id	INT	PK		Benefit ID
membership_type	VARCHAR	FK	NOT NULL	Type
description	TEXT		NOT NULL	Benefit details

23. MEMBERSHIP

Column Name	Data Type	Key	Constraints	Description
membership_id	INT	PK		Membership ID
customer_id	INT	FK	NOT NULL	Customer
membership_type	VARCHAR	FK	NOT NULL	Type
start_date	DATE		NOT NULL	Start date

24. PROMOTION

Column Name	Data Type	Key	Constraints	Description
promo_id	INT	PK		Promo ID
promo_name	VARCHAR	UK	UNIQUE	Name

discount	DECIMAL		> 0	Discount
start_date	DATE		NOT NULL	Start
end_date	DATE			End

25. PRODUCT_PROMOTION

Column Name	Data Type	Key	Constraints	Description
product_id	INT	PK/FK		Product
promo_id	INT	PK/FK		Promotion

26. FESTIVAL

Column Name	Data Type	Key	Constraints	Description
festival_id	INT	PK		Festival ID
festival_name	VARCHAR	UK	UNIQUE	Name
start_date	DATE		NOT NULL	Start
end_date	DATE			End

27. PRODUCT_DEMAND

Column Name	Data Type	Key	Constraints	Description
demand_id	INT	PK		Demand ID
product_id	INT	FK	NOT NULL	Product
festival_id	INT	FK	NOT NULL	Festival
units_sold	INT		≥ 0	Units sold
demand_level	VARCHAR		Low/Medium/High	Demand

28. ROLE

Column Name	Data Type	Key	Constraints	Description
role_id	SERIAL	PK		Unique role ID
role_name	VARCHAR	UK	NOT NULL	Role name (Admin, Staff)
access_level	INT		CHECK ≥ 0	Permission level

29. SHIFT

Column Name	Data Type	Key	Constraints	Description
shift_id	SERIAL	PK		Unique shift ID
shift_name	VARCHAR	UK	NOT NULL	Shift name
start_time	TIME		NOT NULL	Shift start
end_time	TIME		CHECK start < end	Shift end

30. EMPLOYEE

Column Name	Data Type	Key	Constraints	Description
employee_id	SERIAL	PK		Unique employee ID
name	VARCHAR		NOT NULL	Employee name
role_id	INT	FK	NOT NULL	References ROLE
salary	DECIMAL		CHECK ≥ 0	Employee salary
contact	VARCHAR	UK	UNIQUE	Contact number
shift_id	INT	FK	NOT NULL	References SHIFT
hire_date	DATE		DEFAULT CURRENT_DATE	Hiring date

31. USER_LOGIN

Column Name	Data Type	Key	Constraints	Description
user_id	SERIAL	PK		Unique user login ID
username	VARCHAR	UK	NOT NULL, length ≥ 4	Login username
password_hash	VARCHAR		NOT NULL	Encrypted password
employee_id	INT	FK	UNIQUE	Linked employee
last_login_time	TIMESTAMP			Last login
is_active	BOOLEAN			Account status

▪ TRIGGER AND AUTOMATION

Trigger Name	Table	Event	Timing	Description
trg_after_sale_insert	SALES_BILL_DETAILS	INSERT	AFTER	Deducts sold quantity from inventory automatically.
trg_update_loyalty_balance	LOYALTY_TRANSACTION	INSERT	AFTER	Updates loyalty points (earn/redeem) in account.
trg_update_bill_total	SALES_BILL_DETAILS	INSERT, UPDATE	AFTER	Recalculates total bill amount automatically.
trg_low_stock_alert	INVENTORY	UPDATE	AFTER	Logs alert when stock reaches minimum level.
trg_check_expiry_before_sale	SALES_BILL_DETAILS	INSERT	BEFORE	Prevents sale of expired products.

• PROCEDURES AND FUNCTIONS

Name	Type	Parameters	Description
pr_create_restock_order	Procedure	warehouse_id, employee_id	Creates purchase order for low-stock items automatically.
pr_redeem_loyalty_points	Procedure	customer_id, points_to_use	Redeems loyalty points after validating balance.
pr_get_fefo_pick_list	Procedure	product_id	Suggests batch with earliest expiry for picking.
fn_suggest_fefo_batch	Function	product_id	Returns best batch and days until expiry.
pr_apply_category_discount	Procedure	category_id, discount_percent	Applies discount to all products in a category.
pr_purge_expired_stock	Procedure	employee_id	Removes expired stock and logs stock-out transactions.

▪ VIEWS

View Name	Description
VW_INVENTORY_STATUS	Displays product stock details across warehouses including batch and expiry.
VW_LOW_STOCK_ALERTS	Identifies low-stock items and suggests reorder quantity with supplier details.

VW_DAILY_SALES_LEDGER	Provides detailed daily sales transactions including customer, product, and payment info.
VW_CUSTOMER_MEMBERSHIP_STATUS	Shows customer membership details along with loyalty points.
VW_STAFF_SALES_PERFORMANCE	Tracks employee performance based on bills processed and revenue generated.
VW_PRODUCT_PROFIT_MARGIN	Calculates profit per unit and margin percentage for products.
VW_FESTIVAL_DEMAND_PLANNER	Displays product demand trends during festivals for planning.
VW_STOCK_MOVEMENT_LOG	Maintains a log of stock movements for auditing purposes.

▪ RELATIONSHIP,CARDINALITY AND CASCADING

Parent Table	Cardinality	Child Table	ON DELETE	Meaning
ROLE	1 → M	EMPLOYEE	NO ACTION	One role can be assigned to many employees
SHIFT	1 → M	EMPLOYEE	NO ACTION	One shift can be assigned to many employees
EMPLOYEE	1 → 1	USER_LOGIN	CASCADE	One employee has one login; deleting employee removes login
CATEGORY	1 → M	SUB_CATEGORY	CASCADE	One category has many subcategories
SUB_CATEGORY	1 → M	PRODUCT	NO ACTION	One subcategory contains many products
BRAND	1 → M	PRODUCT	SET NULL	One brand has many products; removing brand clears reference
EMPLOYEE	1 → M	WAREHOUSE	SET NULL	One employee can manage many warehouses
PRODUCT	1 → M	INVENTORY	NO ACTION	One product can exist in multiple inventory records
WAREHOUSE	1 → M	INVENTORY	NO ACTION	One warehouse stores multiple inventory items
INVENTORY	1 → M	STOCK_TRANSACTION	NO ACTION	One inventory record can have many transactions
EMPLOYEE	1 → M	STOCK_TRANSACTION	NO ACTION	One employee handles many transactions
SUPPLIER	1 → M	STOCK_TRANSACTION	SET NULL	Supplier reference removed if deleted

PRODUCT	1 → M	REORDER_LEVEL	CASCADE	Reorder rules deleted if product removed
WAREHOUSE	1 → M	REORDER_LEVEL	CASCADE	Reorder rules deleted if warehouse removed
EMPLOYEE	1 → M	SALES_BILL	NO ACTION	One employee processes many bills
CUSTOMER	1 → M	SALES_BILL	SET NULL	Customer can have many bills
SALES_BILL	1 → M	SALES_BILL_DETAILS	CASCADE	Bill details removed if bill deleted
PRODUCT	1 → M	SALES_BILL_DETAILS	NO ACTION	Product appears in many sales
SALES_BILL	1 → M	SALES_RETURN	CASCADE	Returns deleted if bill removed
PRODUCT	1 → M	SALES_RETURN	NO ACTION	Product can be returned multiple times
EMPLOYEE	1 → M	SALES_RETURN	SET NULL	Removes employee reference if deleted
SALES_BILL	1 → M	PAYMENT	CASCADE	Payments deleted if bill removed
PAYMENT	1 → 1	RECEIPT	CASCADE	One payment generates one receipt
SALES_BILL	1 → 1	RECEIPT	CASCADE	One bill has one receipt
CUSTOMER	1 → 1	LOYALTY_ACCOUNT	CASCADE	Each customer has one loyalty account
LOYALTY_ACCOUNT	1 → M	LOYALTY_TRANSACTION	CASCADE	One account has many transactions
SALES_BILL	1 → M	LOYALTY_TRANSACTION	SET NULL	Transaction may link to bill
MEMBERSHIP_TYPE	1 → M	MEMBERSHIP_BENEFITS	CASCADE	Benefits removed if type deleted
CUSTOMER	1 → 1	MEMBERSHIP	CASCADE	Customer memberships deleted if customer removed
MEMBERSHIP_TYPE	1 → M	MEMBERSHIP	NO ACTION	Membership depend on type
PRODUCT	1 → M	PRODUCT_PROMOTION	CASCADE	Product linked to promotions through junction table
PROMOTION	1 → M	PRODUCT_PROMOTION	CASCADE	Promotions linked to product through junction table
PRODUCT	1 → M	PRODUCT_DEMAND	CASCADE	Demand data removed if product deleted
FESTIVAL	1 → M	PRODUCT_DEMAND	CASCADE	Festival demand removed if festival deleted

SUPPLIER	1 → M	PURCHASE_ORDER	NO ACTION	Supplier supplies many orders
EMPLOYEE	1 → M	PURCHASE_ORDER	NO ACTION	Employee manages purchase orders
PURCHASE_ORDER	1 → M	PURCHASE_ORDER_DETAILS	CASCADE	Order details deleted if order removed
PRODUCT	1 → M	PURCHASE_ORDER_DETAILS	NO ACTION	Product appears in many orders
PRODUCT	1 → M	PURCHASE_RETURN	NO ACTION	Product can be returned
PURCHASE_ORDER	1 → M	PURCHASE_RETURN	NO ACTION	Return linked to purchase order
EMPLOYEE	1 → M	PURCHASE_RETURN	NO ACTION	Employee processes returns

▪ **SCHEMA STATISTICS**

Metric	Count
Total Modules	5
Total Tables	31
Total Views	8
Total Triggers	5
Total Procedures	5
Total Functions	1
Total Foreign Key Relationships	40+

QUERIES

- **Top Customers by Total Spending**

```
SELECT c.customer_id, c.name, SUM(sb.total_amount) AS total_spent
FROM CUSTOMER c
JOIN SALES_BILL sb ON c.customer_id = sb.customer_id
GROUP BY c.customer_id, c.name
ORDER BY total_spent DESC
LIMIT 5;
```

- **Most Sold Products**

```
SELECT p.product_name, SUM(sbd.quantity_sold) AS total_sold
FROM PRODUCT p
JOIN SALES_BILL_DETAILS sbd ON p.product_id = sbd.product_id
GROUP BY p.product_name
ORDER BY total_sold DESC;
```

- **Low Stock Products (with Inventory)**

```
SELECT p.product_name, i.quantity_available
FROM PRODUCT p
JOIN INVENTORY i ON p.product_id = i.product_id
WHERE i.quantity_available < 10;

Supplier-wise Total Purchases
SELECT s.supplier_name, SUM(pu.total_amount) AS total_purchase
FROM SUPPLIER s
JOIN PURCHASE pu ON s.supplier_id = pu.supplier_id
GROUP BY s.supplier_name;
```

- **Monthly Sales Report**

```
SELECT DATE_TRUNC('month', bill_date) AS month,
SUM(total_amount) AS total_sales
FROM SALES_BILL
GROUP BY month
ORDER BY month;
```

- **Festival + Promotion Combined Analysis**

```
SELECT f.festival_name, pr.promotion_name, SUM(pd.units_sold) AS sales
FROM FESTIVAL f
```

```
JOIN PRODUCT_DEMAND pd ON f.festival_id = pd.festival_id  
JOIN PRODUCT_PROMOTION pp ON pd.product_id = pp.product_id  
JOIN PROMOTION pr ON pp.promotion_id = pr.promotion_id  
GROUP BY f.festival_name, pr.promotion_name  
ORDER BY sales DESC;
```

CONCLUSION

The Retail Management System database was successfully designed and implemented using PostgreSQL to manage key retail operations such as customers, inventory, sales, payments, and supplier interactions. The database structure was carefully planned and normalized up to Third Normal Form (3NF), ensuring minimal redundancy and high data integrity.

Through the use of well-defined relationships, constraints, triggers, and views, the system is able to handle real-world scenarios such as inventory updates, billing processes, loyalty management, and purchase handling efficiently. The inclusion of advanced modules like profit and marketing analysis further enhances the practical applicability of the system.

Overall, the project demonstrates a scalable and efficient database design that supports organized data management and can be extended in the future with additional features such as user interfaces and real-time analytics.